Incremental implementation packages

Proposal only. No phase below was implemented in production during this design run. The first slice uses existing reconciliation → finance/accounting behavior; it does not start with new billing or a platform-wide rewrite.

Effort ranges are planning estimates in engineering person-days, assuming a developer familiar with this repository plus available product/QA review. They are not calendar promises or estimates based on a confirmed staffing plan. External approval, country review, provider onboarding and environment access can add elapsed time independently. No cloud-price or subscription-price estimate is assumed.

Phase 0 — Make existing access truth reliable

Outcome: administrators can trust the current module status, and the team has a safe basis for configuration changes.

Scope and reuse: existing catalogue, evaluator, protected action/API helpers, activation transaction, UI button and module tests. Correct C01–C07 in a separately authorized implementation task. Preserve existing slugs and routes.

Proposed work: update `services/modules/module-control-contracts.ts`, `module-entitlement.service.ts` and `module-catalog.service.ts`; align mode/reason metadata in guarded callers and the settings page. Add focused regressions in existing module service/action tests. Introduce an explicit legacy-versus-migrated entitlement source; never use array emptiness as both “unknown” and “no grants.”

Dependencies and owner: platform/backend owner with IAM review and product owner for legacy access interpretation. No SaaS billing provider is needed. Human review of migrated entitlement meaning is needed only where source evidence is genuinely ambiguous.

Acceptance: missing organizations fail at the module boundary; expired/not-yet-valid grants cannot authorize work; suspended dependencies cannot supply write capabilities; legitimate read-only queries behave consistently; mode diagnostics match enforced decisions; duplicate grant ambiguity has a defined outcome; legacy reconciliation activation does not silently remove 13 unrelated grants. Unknown legacy values are retained for reconciliation rather than silently discarded.

Verification: focused service/action/API cases, before/after fixtures for empty and populated intent, and wildcard/tenant negative tests. Confirm outer guards and domain checks remain intact. Pure tests can proceed before a database environment is available; real transaction evidence needs an authorized isolated target later.

Migration/rollback: retain legacy adapter behavior only for identified unmigrated organizations; default corrections must not quietly expand access. Snapshot old resolution and compare. Roll back the incompatible new resolver through a scoped policy/configuration rollback, preserving audit and any data already created. Do not restore revoked access as a rollback side effect.

Effort / uncertainty / operating cost: 3–6 days. Main uncertainty is legacy tenant intent interpretation and actual caller coverage. Low new operating cost; requires truthful decision telemetry.

Phase 1 — Prove reconciliation configuration preview and activation

Outcome: an organization can understand and safely add reconciliation with its current finance/accounting dependencies.

Scope and reuse: current `activateTenantModule`, `TenantModuleActivationButton`, reconciliation workbench, module catalogue and the existing action's fresh-auth/permission boundary. Reuse existing business matching/posting services unchanged except for explicit composition adapters where necessary.

Proposed additions: `configuration-planner.service.ts`, a minimal versioned manifest adapter for reconciliation/finance/accounting/core, a read-only preview action, and a review component that shows access added/kept/removed, retained records, dependencies and pending prerequisites. Add an expected baseline/revision token and a stable request ID to submission. A preview is not approval to purchase or activate a production tenant.

Dependencies and owner: Phase 0; platform/backend owner, finance domain owner and frontend/QA owner. Product selects one pilot journey/account scope. For local prototype tests, synthetic data is enough.

Acceptance: preview the existing effective configuration before changing it; reject stale submissions; show every loss of new-work access; preserve legacy rights unless explicitly and validly changed; server recomputes authoritative facts; duplicate submission cannot create two revisions/audits or repeated business effects. An authorized user can reach reconciliation, read the permitted finance evidence and see its accounting source links. A denied user/direct API attempt cannot bypass the policy. Domain financial tests remain unchanged/passing.

Verification: one complete authorized and one denied reconciliation journey, missing dependency, stale preview, duplicate retry and uncertain source retrieval. Validate empty versus error behavior. Include known legacy-wide and explicit tenant fixtures. Use isolated environment data only; capture role, route, source revision, timing and result with private evidence references.

Migration/rollback: retain existing route/button entry, adding preview ahead of the current command. Limit rollout to a named cohort. If the new preview/submit layer fails, disable new configuration changes while maintaining the last verified active capabilities. Do not widen a global enforcement mode to work around a failed test.

Effort / uncertainty / operating cost: 5–9 days after Phase 0. Uncertainty: representing legacy baseline safely without prematurely implementing the complete grant schema. Low incremental runtime cost; preview should read only necessary registry/configuration and scoped obligation summaries.

Concrete first delivery sequence

1. Add regression fixtures for C07 and its related empty/missing-source cases to the existing module tests.
2. Correct legacy derivation/activation preservation and truthful decision metadata.
3. Implement a pure reconciliation plan/impact function against the current catalogue.
4. Expose a server-derived read-only preview and render it before the existing fresh-auth action.
5. Bind submission to the reviewed baseline; reject stale changes and verify duplicate behavior.
6. Demonstrate one reconciliation journey in an authorized isolated environment, with matching source references and no change to unrelated tenant rights.

This is the first practical implementation slice. Estimated combined Phases 0–1: **8–15 engineering days**, subject to the uncertainties above. Evidence collection and design preparation can start without the unavailable commercial/approval decisions listed later.

Phase 2 — Persist desired state, grants and resumable preparation

Outcome: reliable configuration changes survive retries, crashes and concurrent administrators.

Reuse: known manifest/effective-access contracts, domain services, existing audit/outbox/migration practices. Expand `services/modules/` with revision/provisioning services. Add the minimal schema records from CONTRACTS: tenant configuration revision, resolved grant provenance, provisioning steps and lifecycle events. Package/subscription persistence can remain absent until a commercial source is actually selected.

Owner/dependencies: database/migration and platform owners; Phases 0–1; a verified isolated migration/restore target. Integrate with the existing migration control plane and its applicable gates rather than inventing a second deployment process.

Acceptance and verification: tenant-inclusive uniqueness/references; expected revision conflicts; idempotency key/payload binding; resume after process death; active revision does not move before all required steps succeed; source/evidence links remain intact. Backfill reconciles old versus new effective rights per organization and flags ambiguity without silent promotion.

Migration/rollback: additive schema, read-only backfill comparison, selected cohort dual-read, explicit reconciliation, then cutover. Avoid uncontrolled dual writes; if needed temporarily, make one writer authoritative and validate both read models. Keep compatibility columns/reader until rollback evidence allows removal. Test forward and recovery on representative data before applying to an authorized target.

Effort / uncertainty / operating cost: 8–15 days. Data volume, legacy variants and actual migration gates dominate uncertainty. Adds durable job/state storage and monitoring, not a separate service by default.

Phase 3 — Expand the unified shell and capability boundaries

Outcome: retail and service customers receive coherent role workspaces without unrelated screens.

Reuse: sidebar configuration, existing layout/navigation, known permission registry, canonical tables/forms, EN/FR messages, HRIS/accounting/finance services. Add a server-produced navigation/capability read model and adapt existing UI contributions incrementally. Extract the proposed internal integrity interfaces around current ledger services; do not move/duplicate their authoritative records.

Owner/dependencies: frontend, domain and IAM owners; Phase 2 for real state; finance approval of capability boundary meaning. Explicitly map current slugs to proposed IDs. A country-rule engine remains mandatory for applicable activity while its specialist workspace is optional.

Acceptance and verification: retail and service journeys match MODULE_CATALOG, direct-route and transport guards agree, unauthorized source fields cannot leak through reports/analytics, no duplicate route/menu owners, EN/FR and mobile/keyboard checks on real components. HRIS source access remains narrower than general user administration. Build a module-by-surface matrix and resolve current unclassified cases for the cohort.

Migration/rollback: preserve deep links through adapters, version permission mappings and cache keys, switch one shell cohort at a time. Revert presentation independently of the underlying access policy; reverting the shell cannot grant access.

Effort / uncertainty / operating cost: 10–20 days for two configurations, not all 18 selectable modules. Unknown surface ownership/legacy labels are the main uncertainty. Cache scoped read models carefully; avoid dozens of independent entitlement/database checks per rendered row.

Phase 4 — Controlled lifecycle, providers and commercial sources

Outcome: upgrades, suspension, reactivation, provider changes and subscription updates are supportable without losing records or creating duplicate work.

Reuse: configuration history, outbox, provider boundaries, existing AP/payroll/offline controls and native approval-register design. Add operation-level obligation summaries, continuation policy, provider inbox reconciliation, approved package versions and subscription snapshots when an actual billing source is chosen.

Owner/dependencies: platform/SRE, finance/records and commercial owners; verified package/expiry policy; established signing/authority process where actual policy requires decisions. No third-party code marketplace.

Acceptance and verification: open payment/payroll/offline obligations prevent unsafe final deactivation; security revocation takes precedence; commercial expiry preserves only adopted retained-history capabilities; duplicate/out-of-order provider events do not create grant drift or repeat payments; metering counts business usage once. Verify user-readable change history and source-to-provider reconciliation.

Migration/rollback: shadow commercial resolution before it affects access; compare current grants with proposed resolved grants; record conflicts for review. Initially separate quote/purchase acceptance from provisioning. Pause new provider-driven changes on failure, maintain known-safe active state and reconcile actual provider state before retry. Never reverse a payment through configuration rollback.

Effort / uncertainty / operating cost: 12–25 days for one provider and narrow lifecycle cohort; onboarding/external evidence can extend elapsed time. Requires durable workers, useful alerts, quotas and private evidence operations. Additional providers/countries are separate estimates.

Phase 5 — Pilot assurance and measured expansion

Outcome: a named organization/cohort can operate supported workflows with evidence, recovery and a defined owner.

Reuse: existing Astra remediation findings, X1 contracts, release/migration gates, permission tests and operating runbooks. Complete only applicable outstanding prerequisites, bind evidence to the actual build/configuration/route/role scope and carry forward valid unchanged-source evidence with explicit provenance.

Owner/dependencies: release/QA and SRE owners; product/finance/security reviewers for actual required decisions. Native approval register must be established and operating if it is the selected authority source. An unsigned form or blueprint is insufficient.

Acceptance and verification: actual performance meets approved budgets for the chosen routes; scoped role/tenant negatives and real migration/restore checks pass; private artifacts cannot bypass authorization;

installed/CI dependency state is reproducible; alerts reach the operator; recovery is demonstrated. Apply resource cancellation and distributed rate controls for included jobs/agents. Screen-reader and applicable domain/country review remain explicit evidence tasks.

Migration/rollback: staged cohort rollout with clear stop criteria, latest safe active revision, observability and reconciliation. Pilot success does not automatically certify other countries, modules, providers or production scale.

Effort / uncertainty / operating cost: 5–10 days of assurance for a ready narrow cohort, excluding unresolved prerequisite fixes and external review. Ongoing cost is owner time for alerts, reconciliation, backups, customer support and patching; measure volumes before infrastructure changes.

Evidence reuse without unnecessary gates

Use one run index for source/build/configuration hashes, test scope/results, role/environment identifiers and approval references. Reuse a result only when its source/dependency/scope binding is still valid. Recollect stale evidence rather than paste an older green status. Separate local engineering, integration, pilot and production: local work can proceed while a genuine external decision remains pending.

Do not create blanket approval requirements for code refactoring, a local design preview or an already-authorized low-risk tenant change. Do not remove financial, statutory, security or release controls to get a green status. Consolidate duplicate evidence generation and make the exact remaining action visible to its owner.